

FIAP

Domain Driven Design | 2ESPZ | 2026

Sistema de Logística para Entregas

E-commerce | Checkpoint 2

Faculdade	FIAP
Disciplina	Domain Driven Design
Turma	2ESPZ
Professora	Profa. Damiana Costa
Período	04/05 a 10/05 de 2026

Integrantes

Nome	RM
Bernardo Moreira	564103
Pedro Batista	563220
Renan Jordão	560618
Larissa Shiba	560462

Perguntas Discursivas

1. Herança

Como foi utilizada: A herança foi utilizada para criar uma hierarquia entre a classe base **Entregador** e as classes especializadas **Motoboy** e **CarroEntregador**. Da mesma forma, a classe abstrata **Pedidos** serve de base para a classe concreta **PedidoEntrega**.

Problema resolvido: Eliminou a redundância de código. Atributos comuns como nome e veículo ficam centralizados na classe pai, enquanto comportamentos específicos — como o cálculo de frete diferenciado (x1.5 para moto, x2.0 para carro) e o atributo placaMoto — ficam encapsulados nas subclasses. Isso facilita a manutenção e a extensão futura do sistema.

2. Interfaces

Foram criadas duas interfaces no sistema:

Interface	Métodos	Implementada por
Entregável	realizarEntrega(), calcularFrete()	Entregador (abstrata)
IPedidos	obterStatusRastreio(), calcularPrazoEstimado()	Pedidos (abstrata)

Vantagem: A interface **Entregavel** define o contrato para a ação de entrega, garantindo que qualquer novo tipo de entregador (ex: Drone, Bicicleta) siga o mesmo padrão sem alterar o restante do sistema. A **IPedidos** permite que diferentes tipos de pedidos sejam rastreáveis de forma padronizada, desacoplando a lógica de rastreamento da implementação concreta.

3. Classe Abstrata

Papel no sistema: As classes **Entregador** e **Pedidos** foram definidas como abstract. Elas centralizam atributos e comportamentos comuns, ao mesmo tempo em que forçam as subclasses a implementar os métodos específicos de cada tipo.

Por que não poderiam ser classes comuns: No domínio de logística, não existe um "entregador genérico" nem um "pedido genérico". Um entregador deve obrigatoriamente ser de um tipo específico (Motoboy ou CarroEntregador) para calcular o frete correto. A abstração impede que o sistema instancie objetos incompletos, garantindo a integridade das regras de negócio.

4. Sobrecarga e Sobrescrita de Métodos

Tipo	Método	Classe	Descrição
Sobrescrita (Override)	calcularFrete(), realizarEntrega()	Motoboy / CarroEntregador	Cada subclasse redefina o frete e a mensagem de entrega
Sobrecarga (Overload)	atualizarStatus()	Pedidos	Versao simples e versao com observacao adicional

Estrutura do Projeto

Arquivo	Tipo	Responsabilidade
Entregavel.java	Interface	Contrato: realizarEntrega + calcularFrete
IPedidos.java	Interface	Contrato: rastreio + prazo estimado
Entregador.java	Classe abstrata	Base de todos os entregadores
Pedidos.java	Classe abstrata	Base de todos os pedidos
Motoboy.java	Classe concreta	Entregador de moto; frete = dist x 1.5
CarroEntregador.java	Classe concreta	Entregador de carro; frete = dist x 2.0
PedidoEntrega.java	Classe concreta	Pedido com entregador responsável
Main.java	Ponto de entrada	Menu interativo via Scanner